SOFTWARE ENGINEERING & ARCHITECTURE REPORT

AI-Powered Biometric Online Voting System

Project Title:	AI Biometric E-Voting Platform	**Backend Stack:**	Python 3.x, Flask, SQLite3, OpenCV
Architecture:	MVC / WebRTC / Computer Vision	**Security Protocol:**	PBKDF2 Password Hashing, RBAC, WebRTC

INITIAL ADMINISTRATOR ACCESS CREDENTIALS

The system is pre-configured with the primary administrator account for evaluation and testing:
• **Admin Username:** `admin123` | • **Password:** `1234` | • **Email:** `sayyedirfaan263@gmail.com`

1. EXECUTIVE SUMMARY

The **AI-Powered Biometric Online Voting System** is an enterprise-grade digital democratic platform engineered to eliminate voter fraud, prevent double-voting, and streamline election management. By integrating computer vision algorithms via OpenCV with a robust Flask-SQLite backend, the application enforces automated voter facial verification at the point of registration.

Voters can securely register either by uploading a photographic portrait or by capturing a real-time live webcam snapshot through WebRTC stream canvas rendering. The embedded AI computer vision engine normalizes contrast, filters out background false positives, and ensures only valid single-person facial images are granted verified voting rights.

2. TECHNICAL ARCHITECTURE & COMPONENT STACK

Layer	Technology / Library	Core Purpose & Responsibility
Backend Core	Python 3.x, Flask 3.0.3	Routing, session management, authentication, role decorators.
AI Engine	OpenCV (`cv2`), NumPy	Haar Cascade detection, contrast equalization, multi-face suppression.
Database	SQLite3 (Relational)	Transactional integrity, foreign key enforcement, uniqueness constraints.
Security	Werkzeug Security	SHA-256 / PBKDF2 password hashing & verification.
Client Layer	HTML5, Modern CSS3, JavaScript	Responsive UI, WebRTC camera capture, SweetAlert2 dynamic alerts.

3. KEY SYSTEM FEATURES

- **Biometric Facial Detection (OpenCV AI Engine):** Validates human presence, normalizes lighting using histogram equalization, and rejects corrupt or non-face uploads.
- **Dual-Mode Image Ingestion:** Supports both file uploads (PNG/JPG) and real-time live webcam frame capturing via WebRTC.
- **Atomic Single-Vote Constraint:** Database-level `UNIQUE(user_id, election_id)` restriction guarantees that voters cannot cast duplicate votes under any scenario.
- **Role-Based Access Control (RBAC):** Strict route separation between standard Voters and System Administrators using Flask session decorators (`@admin_required`, `@login_required`).
- **Real-Time Analytics Dashboard:** Live aggregate calculations visualizing election turnouts and candidate vote distributions with percentage indicators.

4. DATABASE ERD & SCHEMA DESIGN

The database follows a normalized relational structure designed in SQLite3 with strict foreign key constraints.

Table Name	Primary Columns	Constraints & Key Features
`users`	id, username, email, password_hash, face_encoding_path, is_verified, role	`username` UNIQUE, `email` UNIQUE, `role` IN ('voter', 'admin')
`elections`	id, title, description, status, created_at	`status` DEFAULT 'active' ('active' / 'closed')
`candidates`	id, election_id, name, party, symbol_path	FOREIGN KEY (`election_id`) REFERENCES `elections`(`id`) ON DELETE CASCADE
`votes`	id, user_id, election_id, candidate_id, timestamp	**UNIQUE(user_id, election_id)** - Prevents duplicate voting

5. AI FACIAL VERIFICATION COMPUTER VISION ENGINE

The facial verification engine (`ai_model.py`) executes a 4-stage pipeline when evaluating voter photos:

1. **Grayscale Transformation & Contrast Equalization:** Converts input frames to 8-bit grayscale and applies `cv2.equalizeHist()` to enhance low-light facial details.
2. **Haar Cascade Detection:** Applies `haarcascade_frontalface_default.xml` with tuned hyper-parameters (`scaleFactor=1.1`, `minNeighbors=6`, `minSize=(60,60)`).
3. **Relative Bounding Box Filtering:** Evaluates detected regions against the primary (largest) face area. Secondary bounding boxes smaller than 30% of the primary area are discarded as background noise.

4. **Validation Enforcement:** Ensures exactly one human face is present. Rejects zero-face or multi-person photographs.

```
# Core AI Verification Snippet (ai_model.py)
import cv2, os

class VoterAIVerifier:
    def __init__(self):
        self.face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')

    def detect_and_validate_face(self, image_path):
        image = cv2.imread(image_path)
        if image is None: return False, "Invalid image file."

        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        gray = cv2.equalizeHist(gray) # Histogram Equalization for Light Correction

        raw_faces = self.face_cascade.detectMultiScale(gray, scaleFactor=1.1,
minNeighbors=6, minSize=(60, 60))

        # Filter noise
        faces = [f for f in raw_faces if (f[2]*f[3]) >= (0.3 * max([w*h for (x,y,w,h)
in raw_faces], default=0))]

        if len(faces) == 0: return False, "No human face detected."
        if len(faces) > 1: return False, "Multiple faces detected. Plain background
required."
        return True, "Facial validation successful."
```

6. SECURITY MATRIX & ROUTE AUTHORIZATION

Endpoint Route	HTTP Method	Access Control	Functionality
`/`	GET	Public	Landing page introducing the platform.
`/register`	GET, POST	Public	Voter account creation, camera/file upload & AI verification.
`/login`	GET, POST	Public	Credential checking, password hash verification, session creation.
`/vote`	GET, POST	@login_required	Display active election ballot, process atomic vote submission.
`/result/<id>`	GET	Public / Voter	Real-time percentage breakdown of election results.
`/admin/dashboard`	GET, POST	@admin_required	Create elections, add candidates, toggle status, manage voters.

7. PROJECT DEPLOYMENT & RUNNING INSTRUCTIONS

Follow these steps to run the application in Visual Studio Code or any terminal environment:

1. **Extract and Open Directory:**

```
cd ONLINE-VOTING-SYSTEM
```

2. **Install Dependencies:**

```
pip install -r requirements.txt
```

3. **Initialize Database & Administrator Account:**

```
python create_admin.py
```

Provide custom details or enter the configured credentials (`admin123` / `1234`).

4. **Launch Web Application:**

```
python app.py
```

5. **Access Application:** Navigate to `http://127.0.0.1:5000` in your web browser.

8. VERIFICATION & TEST PLAN

Test Case ID	Target Component	Test Scenario	Expected Result
TC-01	AI Model	Upload photo with single clear face	PASS - User successfully registered
TC-02	AI Model	Upload image without any face (object/blank)	FAIL - "No human face detected" error alert
TC-03	AI Model	Upload photo with multiple people	FAIL - "Multiple faces detected" error alert
TC-04	Vote Route	Voter attempts to vote twice in same election	FAIL - Prevented by SQLite UNIQUE constraint
TC-05	Admin Security	Standard voter accesses `/admin/dashboard`	FAIL - Redirected to login with "Admin authorization required"